

反復ソルバーのオプション体系

DMPlex Taylor-Hood Navier-Stokes ソルバー (PETSc / RBVMS) における線形ソルバー設定の数値

技術文書 | FGMRES / fieldsplit / blockpc / selfp / Kp / inv_tau_c / inv_nu / blk_u_* / blk_p_* | コード実装確認済

1. 解いている問題の構造

各 Newton 反復において、線形系を解く。速度 $\mathbf{u}$ と圧力 $\mathbf{p}$ が結合した、Taylor-Hood 要素 + PSPG 安定化による鞍点型 (saddle-point) のブロック系である。

$$\underbrace{\begin{pmatrix} A & B^T \\ B & -C \end{pmatrix}}_{\mathcal{K}} \begin{pmatrix} \mathbf{u} \\ \mathbf{p} \end{pmatrix} = \begin{pmatrix} \mathbf{f} \\ \mathbf{g} \end{pmatrix}$$

- A : 速度ブロック (時間項 + 移流 + 粘性 + SUPG/LSIC 安定化)
- B : 発散作用素、 B^T : 圧力勾配作用素
- C : PSPG 由来の圧力-圧力ブロック。安定化系であるため $C \neq 0$

この $\mathcal{K}$ を「どう解くか」を制御するのが、以下の各オプションである。オプションは入れ子 (階層) 構造をなし、それぞれ異なる層に作用する。まず全体像を図 1 に示す。外側 FGMRES から前処理、各ブロックの AMG 解法、圧力 Schur 近似まで、各オプションがどの層に作用するかを俯瞰できる。以降の各章は、この図の各層を上から順に詳述するものである。

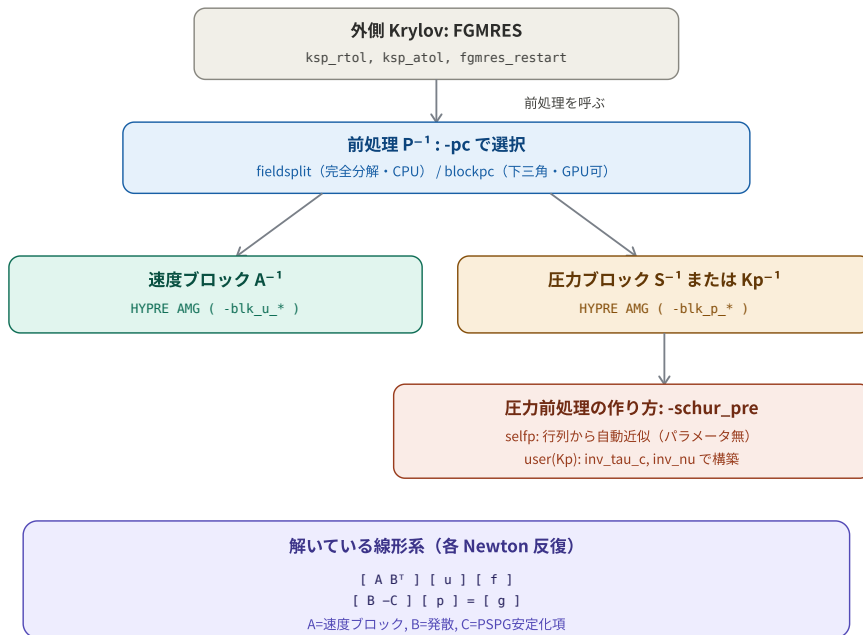


図 1: 反復ソルバーの階層構造 (全体像)。各オプションが作用する層が、上から下へ入れ子になっている。

2. 外側 Krylov 法: FGMRES とその制御

線形系 $Kx = b$ を、FGMRES (Flexible GMRES) で解く。GMRES は k 回目の反復で、解をクリロフ部分空間の中で探索する。

$$x_k \in x_0 + \text{span}\{r_0, Kr_0, K^2r_0, \dots, K^{k-1}r_0\}$$

この部分空間の中で、残差 $\|b - Kx_k\|$ を最小にする x_k を選ぶ。反復を進めるほど残差が小さくなり、ある閾値に達したら停止する。

2.1 停止条件: `ksp_rtol` (相対許容) と `ksp_atol` (絶対許容)

$$\underbrace{\|r_k\| \leq \text{ksp_rtol} \cdot \|r_0\|}_{\text{相対: 初期残差から十分減った}} \quad \text{または} \quad \underbrace{\|r_k\| \leq \text{ksp_atol}}_{\text{絶対: 残差が十分小さい}}$$

どちらかを満たせば停止する。`ksp_atol` が大きすぎると、初期残差 $\|r_0\|$ が既にそれより小さい場合、線形ソルブが 0 反復で「収束済み」と判断され、更新量 $dx = 0$ が返る (itl=0 となり、解が更新されない)。線形ソルバの許容は、Newton が要求する残差レベルと整合させる必要がある。

2.2 `fgmres_restart` (リスタート長)

GMRES は反復ごとに過去の全クリロフベクトルを保持し、それらと直交化 (Gram-Schmidt) する。反復回数 k が増えると、保持ベクトルと直交化コストが $O(k^2)$ で増大する。これを防ぐため、 m 回ごとに、それまでの解を初期値としてクリロフ空間をリセットする (リスタート)。

$$\text{fgmres_restart} = m \quad \Rightarrow \quad m \text{ 回ごとにクリロフ空間を破棄して再出発}$$

- m が大きい: 直交性をよく保ち収束が良いが、メモリ・計算コスト大
- m が小さい: 軽いが、収束が遅くなることがある

FP32 での留意点: FP32 では内積の丸め誤差により、長いリスタートで直交性が崩れやすい。リスタート長を短く (例: 50) すると、直交性の劣化を抑えて安定化できる。

「Flexible」GMRES である理由は、**前処理が反復ごとに変わってもよい**ことを許容するためである。これは、混合精度 (前処理だけ FP32 にする) や、blockpc のような反復的・近似的な前処理を組み込むために重要な性質である。

3. 前処理 P^{-1} : fieldsplit と blockpc

GMRES は、 $\mathcal{K}$ をそのまま解くより、前処理 P^{-1} を施した系を解く方がはるかに速く収束する。

$$P^{-1}\mathcal{K}x = P^{-1}b$$

P が $\mathcal{K}$ に近いほど $P^{-1}\mathcal{K} \approx I$ に近づき、少ない反復で収束する。理想は $P = \mathcal{K}$ だが、それは $\mathcal{K}$ を完全に解くことに等しいので、現実には $\mathcal{K}$ の「近似」を P として使う。**この近似の作り方が、fieldsplit と blockpc の違いである。**

3.1 fieldsplit (完全なブロック分解)

$\mathcal{K}$ のブロック LDU 分解を用いる。

$$\mathcal{K} = \begin{pmatrix} I & 0 \\ BA^{-1} & I \end{pmatrix} \begin{pmatrix} A & 0 \\ 0 & S \end{pmatrix} \begin{pmatrix} I & A^{-1}B^T \\ 0 & I \end{pmatrix}$$

ここで $S = -C - BA^{-1}B^T$ が Schur 補元。完全分解 (SCHUR_FACT_FULL) は、この 3 つの因子すべてを用いて P^{-1} を構成する。 $\mathcal{K}$ をほぼ正確に表すため、**収束が非常に良い (反復 約50回)**。ただし A^{-1} や部分行列抽出が必要で、GPU (cuSPARSE) では `MatCreateSubMatrix` が未対応のためクラッシュする (CPU 専用)。

3.2 blockpc (下三角近似)

完全分解の下三角部分だけを使う近似である。

$$P = \begin{pmatrix} A & 0 \\ B & S \end{pmatrix}, \quad P^{-1}r: \quad z_u = A^{-1}r_u, \quad z_p = S^{-1}(r_p - Bz_u)$$

この $P^{-1}r$ の計算は、前進代入 (forward substitution) として導かれる。前処理とは「 $Pz = r$ を z について解く」ことであり (逆行列を作るのではない)、ベクトルを速度部 z_u と圧力部 z_p に分けて $Pz = r$ をブロック展開すると、

$$\begin{pmatrix} A & 0 \\ B & S \end{pmatrix} \begin{pmatrix} z_u \\ z_p \end{pmatrix} = \begin{pmatrix} r_u \\ r_p \end{pmatrix} \implies \begin{cases} Az_u = r_u & \cdots (1) \\ Bz_u + Sz_p = r_p & \cdots (2) \end{cases}$$

右上ブロックが 0 であるため、式 (1) は z_u だけの式となり、まず $z_u = A^{-1}r_u$ を単独で求められる (ステップ 1)。次に式 (2) に既知となった z_u を代入・移項すると $Sz_p = r_p - Bz_u$ となり、 $z_p = S^{-1}(r_p - Bz_u)$ が求まる (ステップ 2)。「上から順に解ける」のが下三角の利点である。

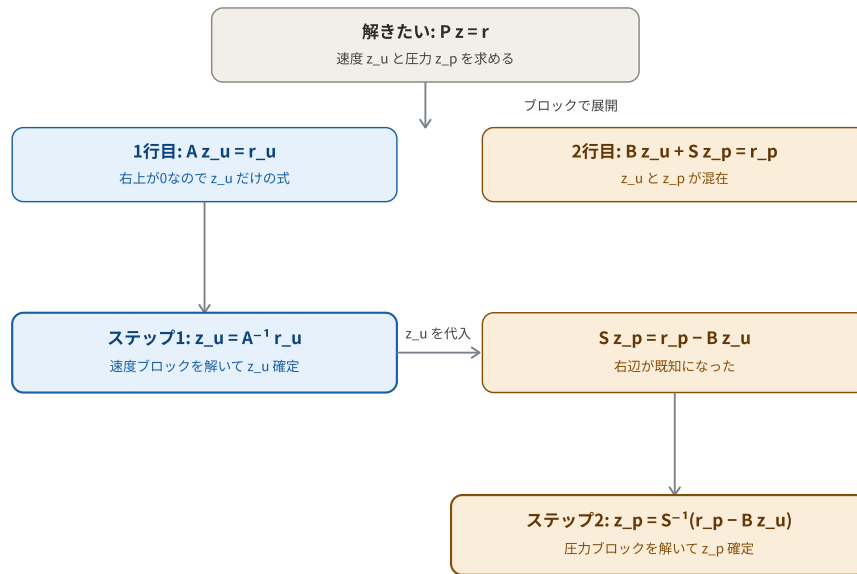


図 2: blockpc における前進代入の流れ。右上ブロックがゼロゆえ z_u を先に確定し、それを用いて z_p を解く。

本来の $\mathcal{K}$ は右上に B^T (圧力勾配) を持つが、blockpc の P はそこを 0 に置き換えている。すなわち $p \rightarrow u$ の逆方向結合 (B^T) を無視する近似であり、それゆえ前進代入で一方通行に解ける。完全分解 (fieldsplit) はこの B^T も上三角因子として扱うため正確で収束が良く、blockpc は B^T を捨てた近似のため収束がやや弱い (反復 約140回)。一方、構造が単純で GPU 対応の形に実装でき、**GPU で使える唯一の選択肢** である。

項目	fieldsplit (完全分解)	blockpc (下三角近似)
右上 B^T の扱い	上三角因子として考慮	無視 (0 に置換)
$\mathcal{K}$ への近さ	ほぼ正確	近似
線形反復回数 (目安)	約 50 回	約 140 回
GPU (cuSPARSE)	不可 (SubMatrix 未対応で SEGV)	可 (PCSHELL 実装)

4. 各ブロックの解法: -blk_u_* と -blk_p_*

fieldsplit も blockpc も、内部で A^{-1} (速度ブロック) と S^{-1} (圧力ブロック) の作用が必要である。これらをそれぞれ HYPRE BoomerAMG (代数マルチグリッド) で近似的に解く。

$$z_u = A^{-1} r_u \Leftarrow \text{AMG で } Az_u = r_u \text{ (-blk_u_*)}, \quad z_p = S^{-1} r_p \Leftarrow \text{AMG で } Sz_p = r_p \text{ (-blk_p_*)}$$

-blk_u_ と -blk_p_ は、それ自体が1つのオプションではなく、**オプション名の頭につくプレフィックス (接頭辞)** である。複数のソルバーが入れ子になっているため、どのソルバーへの設定かを区別する役割を持つ。

```

-ksp_rtol 1e-2      → 外側 FGMRES の rtol
-blk_u_ksp_rtol 1e-3 → 速度ブロックソルバーの rtol (同名でも別物)
-blk_p_ksp_rtol 1e-3 → 圧力ブロックソルバーの rtol
  
```

4.1 ブロックの KSP 設定（どう解くか）

- `-blk_u_ksp_type preonly` : 前処理（AMG）を 1 回適用するだけ（反復しない、最軽量）
- `-blk_u_ksp_type gmres` : ブロックを GMRES で反復的に解く（正確だが重い）
- `-blk_u_ksp_rtol` , `-blk_u_ksp_max_it` : ブロックを解く場合の収束許容・最大反復（preonly では無効）

4.2 ブロックの AMG 設定（調整パラメータの本体）

HYPRE BoomerAMG には、収束性能を左右する多くの調整パラメータがある。AMG は「行列の代数構造から粗グリッドを自動生成し、誤差の各周波数成分を各レベルで潰す」前処理であり、その粗グリッドの作り方・平滑化の仕方を調整できる。

パラメータ	意味と効果
<code>..._strong_threshold</code>	強連結のしきい値。粗グリッド生成でどれだけ強く結合した自由度をまとめるか。 AMG パラメータの中で最も収束に効きやすい 。2D は 0.25 前後、3D は 0.5~0.6 が目安
<code>..._coarsen_type</code>	粗格子化アルゴリズム（HMIS, PMIS, Falgout 等）
<code>..._interp_type</code>	レベル間の補間方式（ext+i 等）
<code>..._relax_type_all</code>	各レベルの平滑化（GPU では Jacobi 系が並列性が高く好まれる）
<code>..._max_levels</code>	マルチグリッドのレベル数

速度ブロック **A**（移流 + 粘性、非対称性が強い）と圧力ブロック **S**（拡散的、素直な楕円型）は性質が異なるため、`-blk_u_*` と `-blk_p_*` で**独立に最適化**できる。

調整の優先順位: (1) 前処理選択（pc, schur_pre）と `inv_tau_c/inv_nu` が性能を大きく支配する。(2) 次に `strong_threshold`。(3) その他 AMG 詳細は最後の詰め。 `-ksp_view` で現在の全ソルバー構成を確認できる。

5. 圧力前処理の作り方: selfp と user(Kp)

ブロック解法に必要な S^{-1} （圧力ブロック）を作るとき、真の Schur 補元 $S = -C - BA^{-1}B^T$ は A^{-1} （密行列）を含むため直接計算できない。そこで S を近似する。その近似法が `-schur_pre` である。

5.1 selfp（パラメータ無しの自動近似）

$$S \approx -B \operatorname{diag}(A)^{-1} B^T$$

A^{-1} を $\operatorname{diag}(A)^{-1}$ （対角の逆数、計算が軽い）で置き換える。その場の行列データだけから作るため調整パラメータが不要で、スケールの心配がない。**FP32 でも安定・最速**であったのは、このパラメータ非依存性による。

5.2 user(Kp)（2 パラメータで構築）

Schur 補元を、物理的知見に基づき、圧力質量行列 M_p と圧カラプラシアン L_p の組み合わせで近似する。

$$K_p = \text{inv_tau_c} \cdot (\tau_M (\mathbf{g} \cdot \mathbf{g})) M_p + \text{inv_nu} \cdot L_p$$

真の Schur 補元は、流れの条件により「質量行列的（時間項・反応的）な性質」と「ラプラシアン的（拡散的）な性質」を併せ持つ。 K_p はその両方を M_p と L_p で用意し、適切な重み（`inv_tau_c` と `inv_nu`）で足し合わせて S を近似する。

5.3 構成要素の定義（コード実装に基づく）

各記号は、有限要素法の基本量である。圧力形状関数を ψ_i とする。

圧力質量行列 M_p （形状関数の積。質量項的・反応的な寄与）

$$(M_p)_{ij} = \int_{\Omega} \psi_i \psi_j d\Omega$$

圧カラプラシアン L_p （形状関数の勾配の積。拡散的な寄与。剛性行列とも呼ぶ）

$$(L_p)_{ij} = \int_{\Omega} \nabla \psi_i \cdot \nabla \psi_j d\Omega$$

安定化パラメータ τ_M （SUPG/PSPG 安定化。時間刻み・移流・粘性から決まる。安定化カーネルと同一式）

$$\tau_M = \left[\left(\frac{\Delta t}{2} \right)^2 + \mathbf{u} \cdot \mathbf{G} \mathbf{u} + C_I \nu^2 (\mathbf{G} : \mathbf{G}) \right]^{-1/2}$$

計量ベクトル $\mathbf{g}$ と内積 $\mathbf{g} \cdot \mathbf{g}$

$\mathbf{g} = (g_0, g_1)$ は2成分の計量ベクトル（Tezduyar系の grid/metric ベクトル）で、各成分は逆ヤコビアン iJ の行和として定義される。

$$g_i = \sum_j \frac{\partial \xi_j}{\partial x_i} = \sum_j iJ_{ij}, \quad \mathbf{g} \cdot \mathbf{g} = g_0^2 + g_1^2 = |\mathbf{g}|^2$$

コード確認結果: K_p 第1項に現れる $\mathbf{g}$ 同士の積は、コード上 $\text{gdotg} = g_0 * g_0 + g_1 * g_1$ という単一の実数（スカラー内積 $\mathbf{g} \cdot \mathbf{g} = |\mathbf{g}|^2$ ）として計算されている。テンソル積 $\mathbf{g} \otimes \mathbf{g}$ （ 2×2 行列）ではない。係数 $\text{inv_tau_c} \cdot \tau_M \cdot (\mathbf{g} \cdot \mathbf{g})$ はスカラーであり、これを圧力質量行列 M_p に掛けて第1項を構成する。

5.4 第1項の係数と τ_C の関係

第1項の係数に現れる $\tau_M \cdot (\mathbf{g} \cdot \mathbf{g})$ は、LSIC/連続安定化パラメータ τ_C の逆数として解釈される。

$$\frac{1}{\tau_C} = \tau_M (\mathbf{g} \cdot \mathbf{g}) \quad \Rightarrow \quad K_p = \text{inv_tau_c} \cdot \frac{1}{\tau_C} M_p + \text{inv_nu} \cdot L_p$$

すなわち K_p の第1項は、LSIC/連続安定化の τ_C を圧力 Schur 前処理に反映したものである。

重要な実装上の差異（コード確認で判明）: τ_C の定義が2箇所で食い違っている。

- 前処理側 (K_p): $1/\tau_C = \tau_M (\mathbf{g} \cdot \mathbf{g})$ 、ここで $\mathbf{g} \cdot \mathbf{g} = \sum_i \left(\sum_j iJ_{ij} \right)^2$ （Tezduyar の $\mathbf{g}$ ベクトル内積形）
- LSIC 安定化側（運動量カーネル）: $1/\tau_C = \tau_M \text{tr}(\mathbf{G})$ 、ここで $\text{tr}(\mathbf{G}) = \sum_i \sum_j iJ_{ij}^2$ （Bazilevs 2007 の trace 形）

$\mathbf{g} \cdot \mathbf{g}$ （行和の二乗和、交差項 $2iJ_{i0}iJ_{i1}$ を含む）と $\text{tr}(\mathbf{G})$ （全成分の二乗和）は一般に異なる値となる。いずれも文献にある正当な定義だが、本ソルバーでは「前処理側 τ_C 」と「安定化側 τ_C 」が別形になっている点に注意。 τ_M の式は両方で完全に同一である。

5.5 2 パラメータの役割

パラメータ	掛かる項	対応する物理
<code>inv_tau_c</code>	$\tau_M (\mathbf{g} \cdot \mathbf{g}) M_p$ （第1項、質量行列系）	時間項・安定化に対応するスケール
<code>inv_nu</code>	L_p （第2項、ラプラシアン系）	粘性拡散に対応するスケール

K_p が真の Schur 補元 S をよく近似すれば収束が良くなるが、それは2パラメータが問題のスケールに合っているかに依存する。FP64で調整した値（例: 1e-3）はFP32ではスケールが合わず発散し、FP32向けに再調整した値（例: 1e-1）にすると blockpc が劇的に改善した（約95秒 → 約8秒）。**パラメータが前処理の質を直接左右し、それが性能を支配する。**

6. 全体の階層構造

各 Newton 反復で、これらが入れ子で動作する。冒頭の図 1 に示した階層を、改めて数式で表すと次のようになる。

$\underbrace{\text{FGMRES}}_{\substack{\text{ksp_rtol/atol} \\ \text{fgmres_restart}}}$ が $\underbrace{P^{-1}}_{\substack{\text{fieldsplit} \\ \text{/ blockpc}}}$ を呼び、内部で $\underbrace{A^{-1}}_{\substack{\text{AMG} \\ \text{blk_u_*}}}$, $\underbrace{S^{-1}}_{\substack{\text{AMG} \\ \text{blk_p_*}}}$ を解く。 S の近似法が $\underbrace{\text{selfp} / Kp}_{\substack{\text{schur_pre} \\ \text{inv_tau_c, inv_nu}}}$

オプション	作用する層	役割
ksp_rtol / ksp_atol	外側 FGMRES	どこまで解くか（停止条件）
fgmres_restart	外側 FGMRES	メモリ・直交化の管理
-pc (fieldsplit/blockpc)	前処理の分解	分解の精密さ（完全 vs 下三角）。 性能を最も支配
-blk_u_* / -blk_p_*	ブロック解法	各ブロックの AMG をどう解くか
-schur_pre (selfp/user)	Schur 近似	圧力 Schur 補元の近似法（何を解くか）
inv_tau_c / inv_nu	Kp の中身	user(Kp) の 2 パラメータ。FP32 では再調整が鍵

層の区別が理解の鍵: inv_tau_c / inv_nu は圧力ブロック行列 K_p そのものを作るパラメータ（「何を解くか」）であり、-blk_p_* はその作られたブロックを AMG でどう解くかのパラメータ（「どう解くか」）である。両者は別の層に作用する独立な調整である。

各オプションを理解する鍵は、「それがこの入れ子構造のどの層に効くか」を意識することである。外側の停止条件 (ksp_*)、前処理の分解精度 (pc)、ブロック解法 (blk_*)、Schur 近似 (schur_pre, inv_*) は、それぞれ別の層で独立に作用する。この層構造を把握すると、「どのパラメータを動かせば何が変わるか」が見通しよく判断できる。

本文書は、DMPlex Taylor-Hood Navier-Stokes ソルバー（PETSc/RBVMs）の反復ソルバー設定に関する技術記録である。 K_p の構成要素および τ_C の定義差異は、ソースコード（BuildPressureKp 関数および安定化カーネル）の実装確認に基づく。